

情報ネットワーク工学入門

第13回

掛下 哲郎

数理・情報部門

kake@is.saga-u.ac.jp

情報ネットワーク工学コースの教育課程

1
年
次

理工学部共通教育
全学教育

2
年
次

コンピュータサイエンス
に基づく専門基礎教育

- ・ プログラミング概論・演習
- ・ データ構造とアルゴリズム
- ・ コンピュータアーキテクチャ
- ・ 情報システム実験 等

3
年
次

情報ネット
ワーク

情報セキュ
リティ

ソフトウェア
工学

ソフトウェア協
同開発実験

4
年
次

卒業研究(研究室配属は3年次後期)

修
士
課
程

情報ネット
ワーク
関係科目群

ソフトウェア
クリエイタ
育成科目群

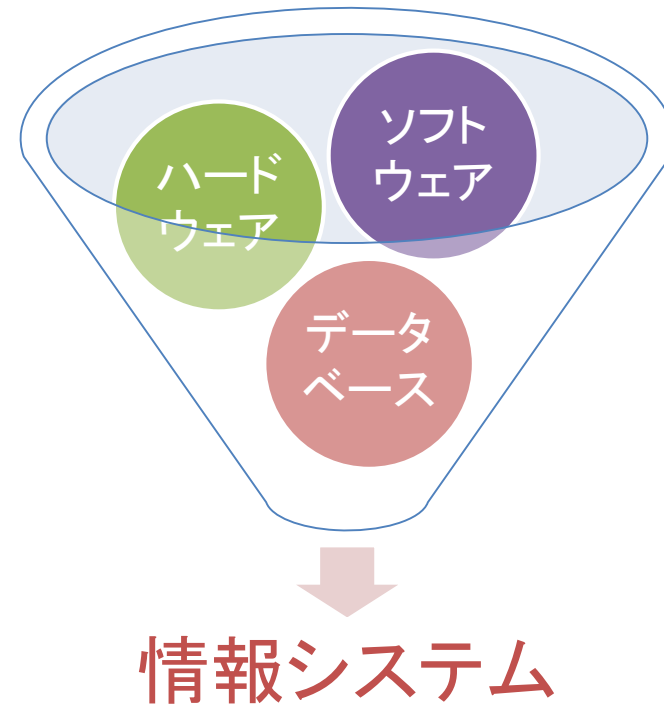
特別研究

第13回の内容

- ソフトウェアの基本的性質
- ソフトウェア工学の目的
- ソフトウェア開発の基本手順
- デジタルトランスフォーメーション(DX)
 - 企業のデジタル化
 - 開発体験

情報システムはあらゆるところに

- ポータルサイト(例: Yahoo! Japan, Google)
- 銀行オンラインシステム
- 電子図書館
- 入試情報システム
- 教務情報システム
- 就職情報システム
- 医療情報システム
- POS(Point of Sales)情報システム
- 電子商取引
- 電子政府, 電子県庁, 電子市役所



ソフトウェアの基本的性質(1)

- 情報システム開発コストの90%以上はソフトウェア開発コスト
 - ハードウェアにはOS, ネットワークを含む.
- 業務で使用するソフトウェアには責任が伴う.
 - 間違いは許されない.
 - 通常はソフトウェア開発者と使用者が異なる.
 - 完全な説明文書とQ and A体制が必要
 - 安定運用(信頼性)が不可欠

スルガ銀-IBM裁判で両社 ×

itpro.nikkeibp.co.jp/article/COLUMN/20131226/52734 ☆

News & Trend 日経コンピュータ

スルガ銀-IBM裁判で両社が上告

争点はITベンダーの「中止提言義務」

2014/01/07
(浅川 直輝 = 日経コンピュータ)

勘定系システムの開発中止の責任を巡り、スルガ銀行が日本IBMに約116億円の支払いを求めた裁判で、上告審における両社の主張が明らかになった。「ITベンダーはユーザー企業に対し、必要に応じてプロジェクトの抜本的な見直し、または中止を提言する義務を負う」。東京高等裁判所がITベンダーに課した新たな義務の是非が、最高裁判所で争点の一つになりそう。

東京高裁が提示した新たな義務とはどのようなものか。過去2回の判決をひもときながら説明する(図1)。

図1 上告審におけるスルガ銀行・日本IBMの主張
互いに「第二審の判決は不当」と譲らず

	第一審判決の賠償額	第二審判決の賠償額
最終合意前	74.1億円	41.7億円

賠償を認定せず

賠償すべき

スルガ銀行の主張

- プロジェクト頓挫の原因が日本IBMの不法行為だった以上、最終合意前の損害もその不法行為が原因とみるべき
- Corebankによる邦銀勘定系システムの開発は「費用対効果の面で不可能」

賠償を認定

賠償は不要

日本IBMの主張

- 最終合意時点で、スルガ銀側もリスクは認識していた。中止提言などの義務をITベンダーだけに負わせるのは不当
- 最終合意時は、まだ「抜本的な見直し」「中止」の提言は必要でなかった

[画像のクリックで拡大表示]

スルガ銀-IBM裁判では、システム開発においてITベンダーが負う義務を指す「プロジェクトマネジメント(PM)義務」の解釈が、判決を左右する大きなポイントとなった。

東京地方裁判所は2012年3月、プロジェクトの企画・提案段階で、勘定系パッケージ「Corebank」の機能検証が不足していたことが、日本IBMのPM義務違反に当たるとして、日本IBMに約74億円の支払いを命じる判決を言い渡した。一方で東京高裁は2013年9月、日本IBMのPM義務違反は、パッケージを選定した提案・企画段階ではなく、要件定義を経て両社が最終合意書を交わした段階で起きたとして、賠償額を約42億円に減額する判決を下した。

なぜ、システム開発は必ず ×

itpro.nikkeibp.co.jp/article/COLUMN/20140130/53358 ☆

新刊・近刊 SYSTEMS

なぜ、システム開発は必ずモメるのか？

49のトラブルから学ぶプロジェクト管理術

2014/02/18
(島津 忠承 = 日経SYSTEMS)

システム開発プロジェクトに失敗はつきもの。では、失敗はなぜ発生し、どうすれば防げるのか。その答えを、裁判に至った事例を基に解説するのが本書だ。著者はNECソフトや日本IBMに勤務していた元ITエンジニアで、現在は東京地方裁判所の民事調停委員(IT事件担当)を務める。その特異な経歴を生かした一冊といえる。

取り上げた事例は49個あり、「要件定義」「プロジェクト計画と管理」「設計」「プログラミング」「テスト」「契約と仕事の完成」というプロジェクトの流れに沿って分類した。このうち例えば要件定義の事例では、「ベンダーが勝手に性能要件を消してしまった」「もし、要件定義の担当者がベンダーと『グル』だったら？」といった事例を挙げる。個々の事例については、美人弁護士とその友人たちの会話という体裁で、概要と防止策をコンパクトに示している。裁判の内容を詳しく知りたい人には物足りないかもしれないが、法律知識に明るくなくても理解できる。

なぜ、システム開発は必ずモメるのか？
49のトラブルから学ぶプロジェクト管理術
細川 義洋 著
日本実業出版社発行
2100円(税込)

「契約・要件定義」から「テスト・検証」までの現実的なソリューション

■関連する記事を読む

- なぜ、システム開発は必ずモメるのか？ (日経ストラテジー, 2013年12月16日)
- システム開発がもめる理由 (ITpro, 2013年12月2日)
- なぜ、システム開発は必ずモメるのか？ (日経コンピュータ, 2013年12月2日)

代表的なソフトウェアの規模

薄型テレビ	60万行
HDD内蔵DVDレコーダ	100万行
通信機能搭載型カーナビ	300万行
第三世代携帯電話	500万行
Microsoft Windows 3.1	300万行
Microsoft Windows 95	1100万行
Microsoft Windows 2000	6000万行
Linux (RedHut 2002)	3000万行



出典: 情報サービス・ソフトウェア産業維新, 産業構造審議会 情報経済分科会
 情報サービス・ソフトウェア小委員会 中間とりまとめ, 2006年6月

ソフトウェアの基本的性質(2)

- ソフトウェア開発のほとんどは人手で実施
- 大規模ソフトウェアが多い。
 - 背景: 同一価格のハードウェアの性能は5年で10倍
 - 一人のプログラマが詳細を把握できるソースコード行数は高々1万行程度
 - チームでの作業分担が必要
 - 担当者間での情報(各担当者の意図)伝達が不可欠

ソフトウェアの基本的性質(3)

- ソフトウェア開発には専門知識が不可欠
 - ソフトウェア工学, プログラミング言語, アルゴリズムとデータ構造, コンパイラ, オペレーティングシステム
 - 構築対象業務に関する知識等
 - 情報処理技術者試験
 - 高スキルのソフトウェア技術者は1000万円/人年
- ソフトウェア開発費用は非常に高額
 - 例: 銀行のオンラインシステム開発は100億円以上
 - セブンイレブンのPOSシステム開発は600億円
 - ソフトウェア開発費用の見積りが重要
 - 既存ソフトウェアの再利用が重要

日本と世界の長者番付(2025年度版)

世界の長者番付

- 1位 イーロン・マスク／3420億ドル(米国／Tesla, Space-X)
- 2位 マーク・ザッカーバーグ／2160億ドル(米国／Facebook)
- 3位 ジェフ・ベゾス／2150億ドル(米国／Amazon)
- 4位 ラリー・エリソン／1920億ドル(米国／Oracle)
- 5位 ベルナール・アルノー／1780億ドル(フランス／LVMH)
- 6位 ウォーレン・バフェット／1540億ドル(米国／Berkshire Hathaway)
- 7位 ラリー・ページ／1440億ドル(米国／Google)
- 8位 セルゲイ・ブリン／1380億ドル(米国／Google)
- 9位 アマンシオ・オルテガ／1240億ドル(スペイン)
- 10位 スティーブ・バルマー／1180億ドル(米国／Microsoft)

日本の長者番付

- 30位 柳井 正／451億ドル(ファーストリテイリング)
- 60位 孫 正義／294億ドル(ソフトバンク)
- 102位 滝崎 武光／195億ドル(キーエンス)
- 540位 重田 康光／64億ドル(光通信)
- 767位 高原 豪久／47億ドル(ユニ・チャーム)
- 789位 安田隆夫／46億ドル(ドン・キホーテ)
- 823位 三木谷 浩史／44億ドル(楽天)
- 868位 森章／42億ドル(森トラスト)
- 902位 三木正浩／40億ドル(ABCマート)
- 1015位 野田順弘／36億ドル(オービック)

ソフトウェア会社のビジネス

Microsoft
の事例

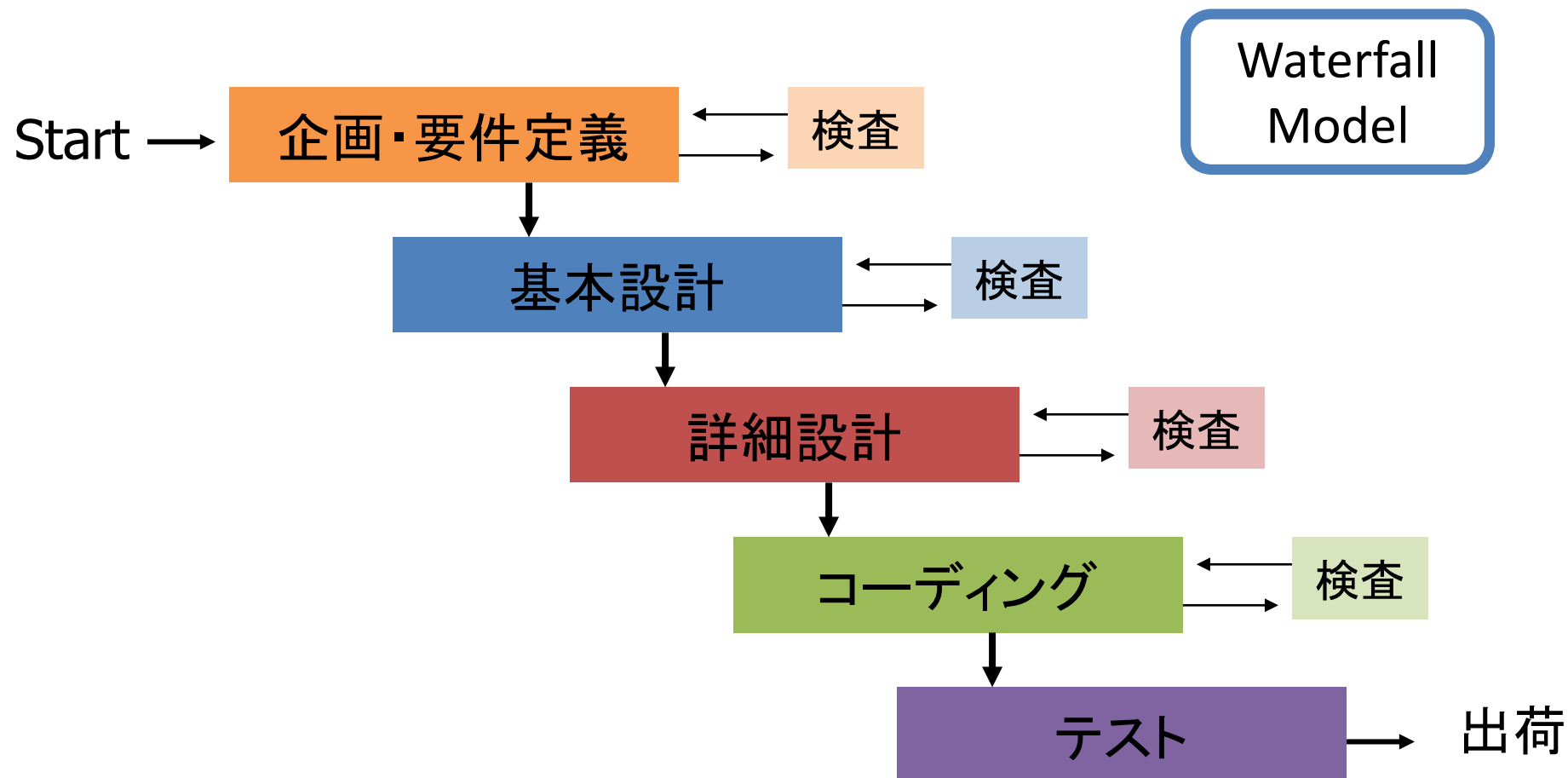
- Windowsの開発費: 100億円以上
 - 一般的な感覚では、極めて高額
- Windowsの販売価格: 1万円程度
- Windowsの販売数: 1億本
 - ソフトウェアのコピーは、ほぼ無料でできる.
 - 何本売っても開発費用はほとんど同じ.
 - インターネットを活用すれば、世界中に販売できる.
- Windowsの売り上げ: 1兆円
 - 開発費の100倍！
- だからMicrosoftは大企業に
 - Bill Gatesは世界有数の大富豪に
- Google, Apple, Facebook, Amazon等も同様

利益: 9900億円

ソフトウェア工学の目的

- 系統的なソフトウェア開発
 - 費用・期間の見積もりを可能にする。
 - 計画的なソフトウェア開発
 - 高品質ソフトウェアの開発
- ソフトウェア開発コストの低減
 - 仕様変更が容易
 - バグの発見と修正が容易
 - 再利用性が高い etc.

系統的なソフトウェア開発手順



企画

- 顧客の現状を分析する。
- 顧客の抱える問題点を理解する。
 - 顧客ニーズ(必要度):この問題が解決できないと何が困るか？
 - 顧客ウォンツ(欲求):この問題が解決できると何が嬉しいか？
- 顧客の問題を解決するための可能な手段を列挙する。
- 列挙された手段の中から最も適切なものを選択する。
 - 顧客ニーズに基づく選択
 - 開発者の強みに基づく選択
 - 開発コスト(費用・納期)に基づく選択

ソフトウェアの企画

問題を見つける

問題を定義する

問題の原因を探る

問題を解決するアイデアを探す

アイデアを改善する

要件定義

- ソフトウェアに対する要求事項を文書化する。
 - 原則として仕様書に書かれている事項だけを実装する。
- 仕様書の記載に漏れがないようにする。
- 仕様書の記載に矛盾がないようにする。
- 開発コストを見積もる。

プロジェクトが大失敗する原因

1. 見積もりミス

- 開発者が関与しない見積もり
- 仕様確定前の見積もり
- 仕様変更後に再見積もりを実施しない。

2. 仕様が確定できないこと

- 仕様の抜けは仕様関係の不具合の中で修正が最も難しい。

ソフトウェア仕様書の事例

第12回 Excelによる仕様化テンプレート - Excel

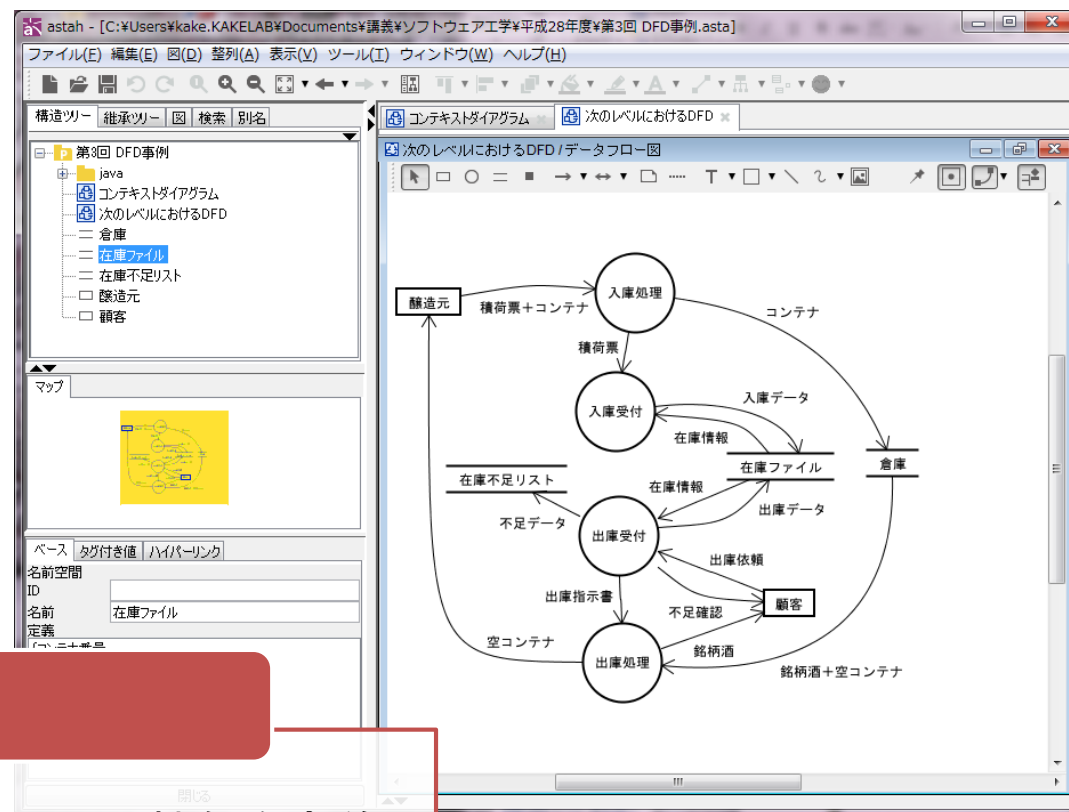
データフロー図(ロ)

	A	B	C
1	要件	MAL01	実行環境および開発環境
2		理由	できるだけ多数の回答を集め、迅速に分析結して開発することを要求する。また、これによ目的とする。
3	説明		データ収集・分析システムは、Webアプリケーション
4	要件	MAL01-01	実行環境はLinux、MySQL、Apache
5	理由		研究グループのWebサーバで
6	説明		Linux、MySQL、Apacheは20
7		☐ MAL01-01-01	Linuxは2010年度時点で最
8		☐ MAL01-01-02	MySQLは2010年度時点で最
9		☐ MAL01-01-03	Apacheは2010年度時点で最
10	要件	MAL01-02	開発環境はPHP、MySQL、
11	理由		委託者側でのソフトウェア改
12	説明		PHP等も2010年度時点で最
13		☐ MAL01-02-01	PHP等は2010年度時点で最
14	要件	MAL02	利用者によるアカウント管理機能
15	理由		・回答に含まれる個人情報保護する。 ・回答者が最新データをいつでも入力できるようにする。 ・管理者側でのデータ編集作業の負担を低減する。
16	説明		利用者(=回答者)が自らアカウント情報を入力・編集できるようにする。
17	要件	MAL02-01	アカウントの新規作成
18	理由		
19	説明		アカウント名はメールアドレスとする。新規作成時に回答者種別を指定させ、それに応じてプロフィール情報の項目を確定する。
20		☐ MAL02-01-01	利用者から、メールアドレス、パスワード、氏名、所属、回答者種別、特記事項を入力する。入力したデータはMySQLテーブルに保存する(管理者フラグ off、確認済みフラグ off、登録日時はシステムクロックより取得して設定)。
21		☐ MAL02-01-02	利用者の所属機関はMySQLテーブルに保存し、その中から選択させるのを原則とする。選択肢に含まれない場合のみ自由記述を認めるが、入力された値は新たな選択肢としてテーブルに追加する。
22		☐ MAL02-01-03	指定可能な回答者種別はMySQLテーブルに保持する。
23		☐ MAL02-01-04	回答者種別に依存する追加項目はMySQLテーブルに保持し、それに従って利用者に値の入力を求める。入力した値はMySQLテーブルに保存する。
24		☐ MAL02-01-05	入力されたメールアドレスに登録確認メールを送信して利用者に確認を求める。確認のために、ユーザー毎にユニークなWebアドレスにアクセスさせる。
		☐ MAL02-01-06	登録確認メールに記載したWebアドレスがメール送信後24時間以内にアクセスされた場合、MySQLテーブルに保持した登録済みフラグをonにし、登録完了メ

準備完了

データフロー図 (DFD)

- ソフトウェア内部でのデータの流れを表現
- ソフトウェアの利用者とソフトウェアの間のデータの流れを表現



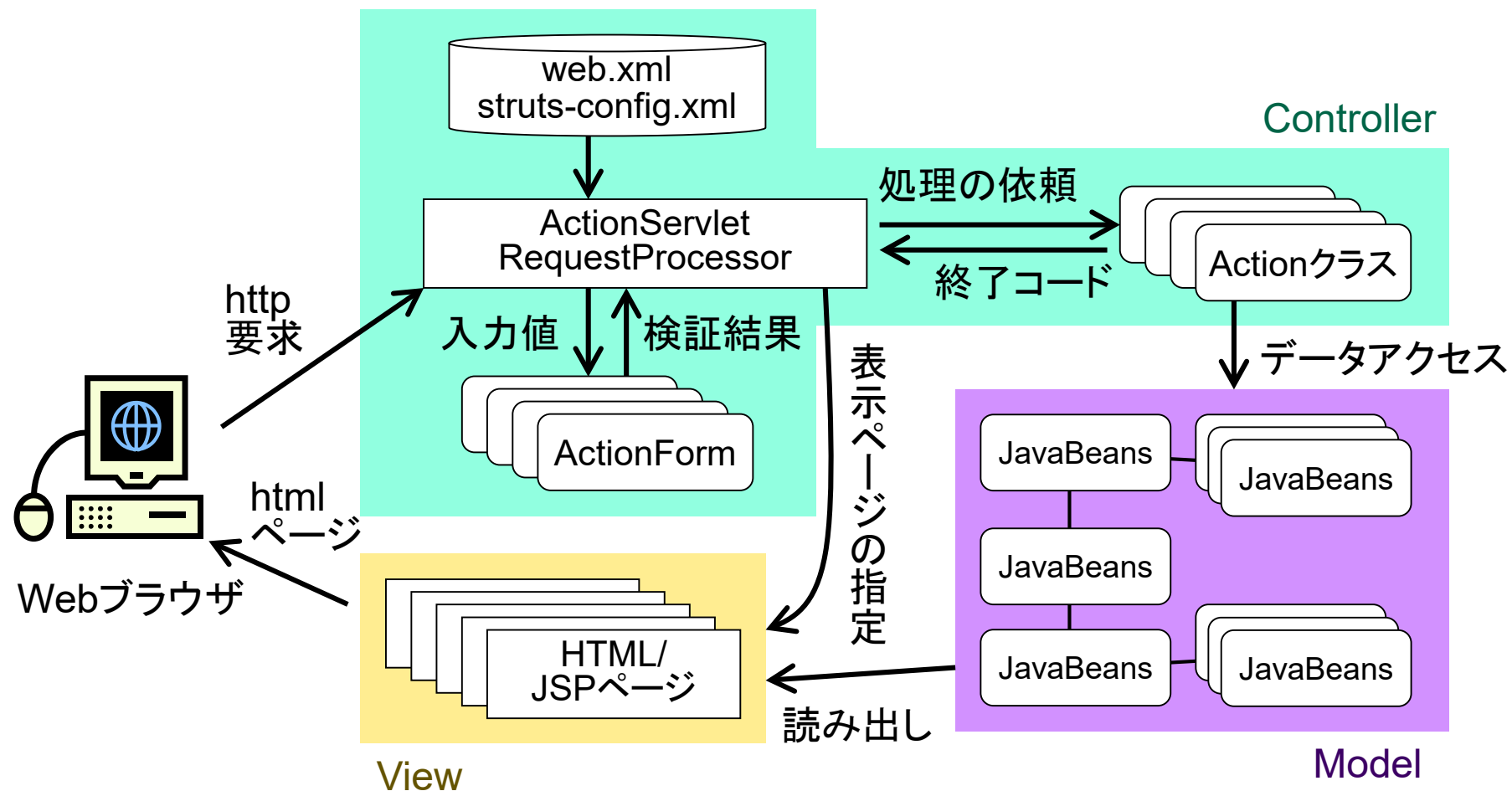
仕様書

- ソフトウェアの機能を定義
- ソフトウェアが満たすべき品質要件を定義

基本設計

- ソフトウェアをいくつかの部分に分割する。
 - 機能に基づいた分割 : 構造化設計
 - データに基づいた分割: オブジェクト指向設計
- モジュール毎に、提供する機能(ルーチン)とその仕様を決める。
 - モジュール名、モジュール間の相互関連
 - ルーチン名、引数・戻り値の名前とデータ構造
- **ソフトウェアの基本構造**を決める重要なステップ

ソフトウェアの設計事例



詳細設計

- モジュールの各機能を実現する手順(**アルゴリズム**)の決定
- アルゴリズムが必要とする**データ構造**の決定
- アルゴリズム設計までは、プログラミング言語とは独立して**自然言語**で行う.

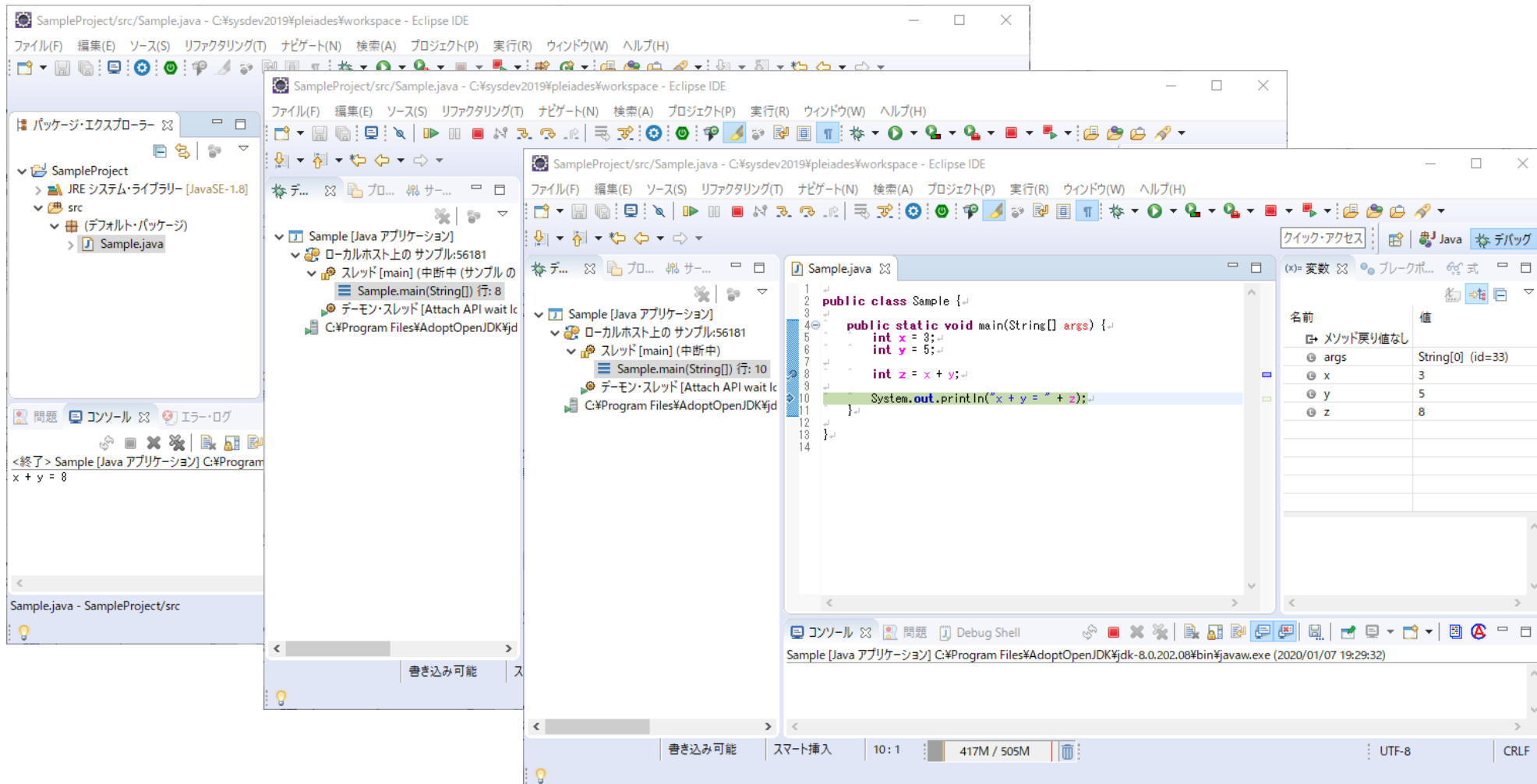
設計の重要性

- ソフトウェア設計は、ソフトウェアの保守性や再利用性に大きく影響する。
- ソフトウェア設計において、ベストの解が1つしかないことは、通常ありえない。
- ソフトウェアの設計は、複雑で反復が必要なプロセスである。
 - 最初に考えついた設計方式が間違っている可能性は高い。
- ソフトウェアの処理効率には、良いコーディングよりも良い設計が大きく影響する。

コーディング

- コーディングとは？
 - 開発に用いるプログラミング言語の決定
 - モジュール設計及び詳細設計に従って、ソフトウェアをプログラミング言語で実現する。
- コーディングの手順
 - プログラムの宣言部を書く。
 - アルゴリズムの各行を高水準のコメントとする。
 - それぞれのコメントの下に対応するコードを埋め込む。
 - アルゴリズムとコードが一貫しているか確認する。
 - 以上の作業を確認する。

Eclipseを用いたコーディング事例



デバッガを用いたステップ実行と変数値の確認

コーディングの重要性

- 素直なコーディングはデバッグ・保守に要するコストを最小化する。
 - ソフトウェア開発のライフサイクルでは、エラー除去に最も時間がかかる。
- 素直なコーディングは高速化をもたらす。
- 高水準プログラミング言語、最適化コンパイラ、最新ハードウェアの組み合わせは、アセンブリ言語プログラムよりも高速

Findbugsを用いたコードの検査

The screenshot displays the Jenkins web interface. The breadcrumb trail indicates the path: Jenkins > 2019年度グループ演習 > Cherry2019 > #22 > FindBugs警告. The left sidebar contains navigation links such as 'プロジェクトへ', '状態', '変更履歴', 'コンソール出力', 'ブレンデキス', '説明の編集', 'Delete build '#22'', 'Git Build Data', 'No Tags', 'Checkstyle警告', 'FindBugs警告', 'テスト結果', and '前のビルド'. The main content area shows the 'FindBugs警告' section for the file 'LoginProcessorTest.java' under the category 'BAD_PRACTICE'. The title is 'カテゴリ BAD_PRACTICE - File LoginProcessorTest.java' and the section is '要約'. The code snippet shows a test method 'testCounter()' and a test method 'testFailedLogin()'. The 'testFailedLogin()' method contains a call to 'file.delete()' which is highlighted in orange. Below the code, a message states: 'RV: java.io.File.delete() の例外的戻り値を無視しています。sysdev.LoginProcessorTest.testFailedLogin()' and explains that the method does not check for exceptions and returns false, which is not the intended behavior for the test.

Jenkins

Jenkins

Jenkins

プロジェクトへ

状態

変更履歴

コンソール出力

ブレンデキス

説明の編集

Delete build '#22'

Git Build Data

No Tags

Checkstyle警告

FindBugs警告

テスト結果

前のビルド

プロジェクトへ

状態

変更履歴

コンソール出力

ブレンデキス

説明の編集

Delete build '#22'

Git Build Data

No Tags

Checkstyle警告

FindBugs警告

テスト結果

前のビルド

カテゴリ BAD_PRACTICE - File LoginProcessorTest.java

要約

```
125 @Test
126 void testCounter() throws IOException {
127     LoginProcessor sut = new LoginProcessor();
128     assertEquals(true, sut.checkUser("abcc", "aaAA11bb"));
129     UserDataReader udr = new UserDataReader("user/abcc.txt");
130     assertEquals("1", udr.getValue("count"));
131 }
132
133 /**
134  * 作成済みであるユーザID:abcc、
135  * 正しくないパスワード:aaaaAA111を入力し、
136  * 警告文が保持され、falseが返ることを確認する。
137  * @throws IOException 例外
138  */
139 @Test
140 void testFailedLogin() throws IOException {
141     File file = new File("user/abcc.txt");
142     file.delete();
143     LoginProcessor sut = new LoginProcessor();
```

RV: java.io.File.delete() の例外的戻り値を無視しています。sysdev.LoginProcessorTest.testFailedLogin()

このメソッドはチェックされていない値を返しています。戻り値は異常が予想外の実行結果を示す可能性があるためチェックするべきです。たとえば、File.delete() はファイルをうまく削除できなかったなら、例外をスローするのではなく false を返します。結果をチェックしないなら例外的戻り値を返すメソッドの呼び出しで予想外の振る舞いの含図に気づきません。

```
148 }
149 }
150 }
```

ソフトウェアテストの目的

- ソフトウェアテストとは？
 - ソフトウェアに入力データ(テストケース)を与えて実行する.
 - 仕様書に基づく実行結果
 - 実行結果AとBを比較
- 意味のあるソフトウェアは必ずバグを含む。
 - ソフトウェアにバグ(誤り)がないことは、一般には示せない。
- 運用前にできるだけ多くのバグを発見して修正するためにテスト等を行なう。
 - ソフトウェアにバグがないことを示すために行なうのではない。

実行結果A

実行結果B

一致しない場合,
バグ発見

JUnitを用いたソフトウェアテスト

The screenshot shows the Jenkins web interface for a project named "Sunflower2019". The top navigation bar includes the Jenkins logo, a search bar, and links for "管理者" (Admin) and "ログアウト" (Logout). The breadcrumb trail indicates the current location: Jenkins > 2019年度グループ演習 > Sunflower2019 > #36 > テスト結果.

The main content area displays the test results for the build. The test "sysdev.UserDataReaderTest.testCount()" (from JUnit Jupiter) has failed. The failure message is "失敗" (Failure). The stack trace shows the error occurred in the "testCount()" method of "sysdev.UserDataReaderTest".

失敗
sysdev.UserDataReaderTest.testCount() (from JUnit Jupiter)

過去 17ビルド 失敗 (🟡 #20 以降)
所要時間 3 ms
📄 説明を記入

スタックトレース

```
java.lang.AssertionError
    at org.junit.Assert.fail(Assert.java:86)
    at org.junit.Assert.fail(Assert.java:95)
    at sysdev.UserDataReaderTest.testCount(UserDataReaderTest.java:126)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at org.junit.platform.commons.util.ReflectionUtils.invokeMethod(ReflectionUtils.java:675)
    at org.junit.jupiter.engine.execution.MethodInvocation.proceed(MethodInvocation.java:60)
    at
org.junit.jupiter.engine.execution.InvocationInterceptorChain$ValidatingInvocation.proceed(InvocationInterceptorChain.java:125)
    at org.junit.jupiter.engine.extension.TimeoutExtension.intercept(TimeoutExtension.java:132)
    at org.junit.jupiter.engine.extension.TimeoutExtension.interceptTestableMethod(TimeoutExtension.java:124)
    at org.junit.jupiter.engine.extension.TimeoutExtension.interceptTestMethod(TimeoutExtension.java:74)
    at
org.junit.jupiter.engine.execution.ExecutableInvoker$ReflectiveInterceptorCall.lambda$ofVoidMethod$0(ExecutableInvoker.java:115)
    at org.junit.jupiter.engine.execution.ExecutableInvoker.lambda$invoke$0(ExecutableInvoker.java:105)
    at
org.junit.jupiter.engine.execution.InvocationInterceptorChain$InterceptedInvocation.proceed(InvocationInterceptorChain.java:116)
```

ソフトウェア工学技術(その1)

プロセス モデル	ソフトウェアの開発工程を表現する。 例: ウォーターフォール開発, アジャイル開発, Scrum, XP
計算 モデル	ソフトウェアを設計するための基本方針を与える。 例: 構造化技法, オブジェクト指向技法
情報 モデル	ソフトウェアが処理対象とする情報の構造を表現する。 例: リレーショナルモデル, ERモデル, オブジェクト指向モデル

ソフトウェア工学技術(その2)

コストモデル	ソフトウェアの規模を見積もる技術 例: FP法, COCOMO
方法論・概念	ソフトウェアの設計原理 例: 段階的詳細化, 情報隠ぺい, ADT
テスト技法	ソフトウェアのバグを減らすための技術 例: ブラックボックステスト, ホワイトボックステスト, ユニットテスト, 結合テスト, レビュー, インспекション
プロジェクト管理技法	多人数によるソフトウェア開発を効果的に進めるための技術

DevOpsとは？



ソフトウェアの開発担当と導入・運用担当が密接に協力する体制を構築し、ソフトウェアの導入や更新を迅速に進める

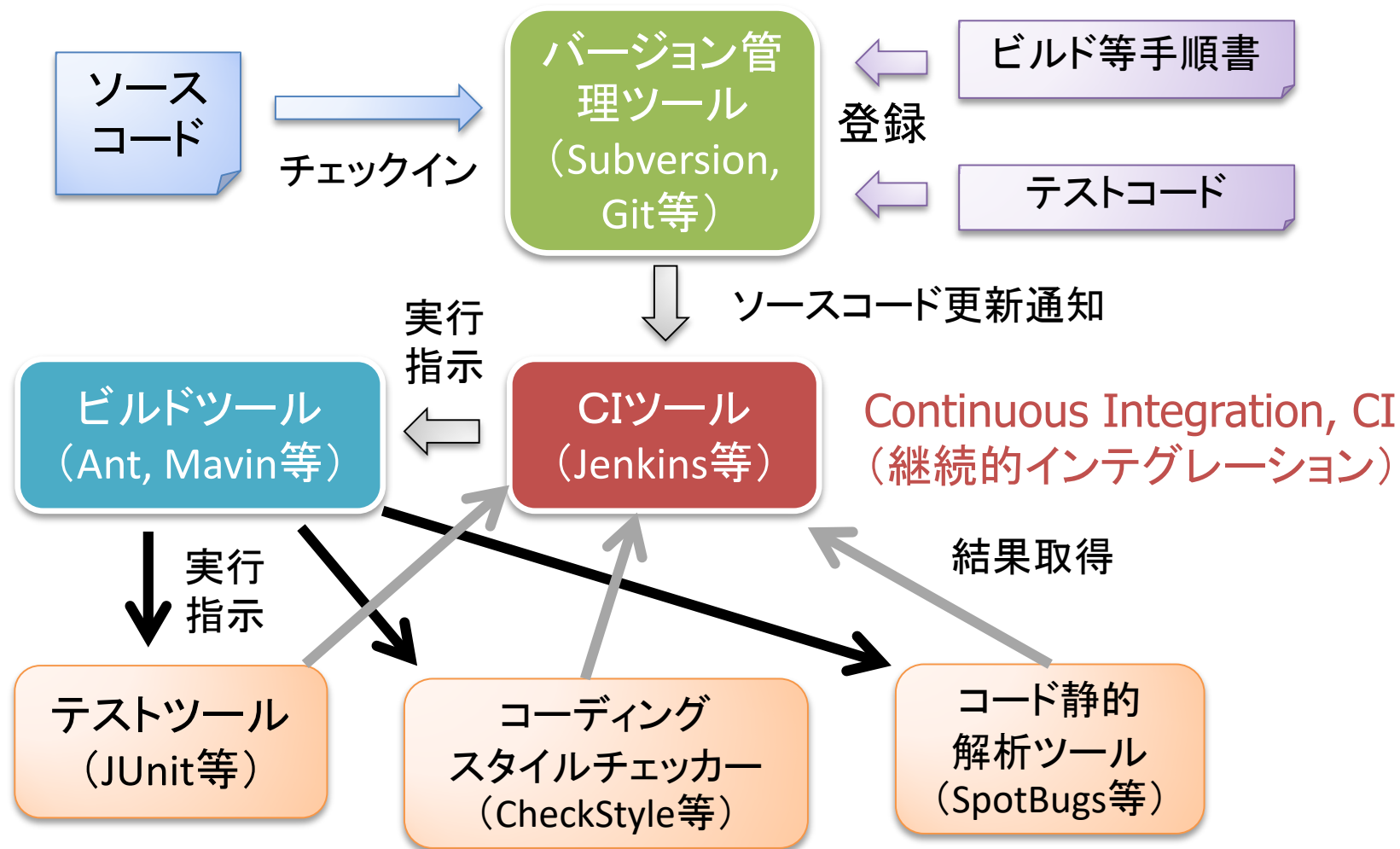
Build

- プログラムをコンパイル, リンクして実行形式に変換する

Deploy

- ソフトウェアが運用できるように, 実行環境にインストール, 設定する

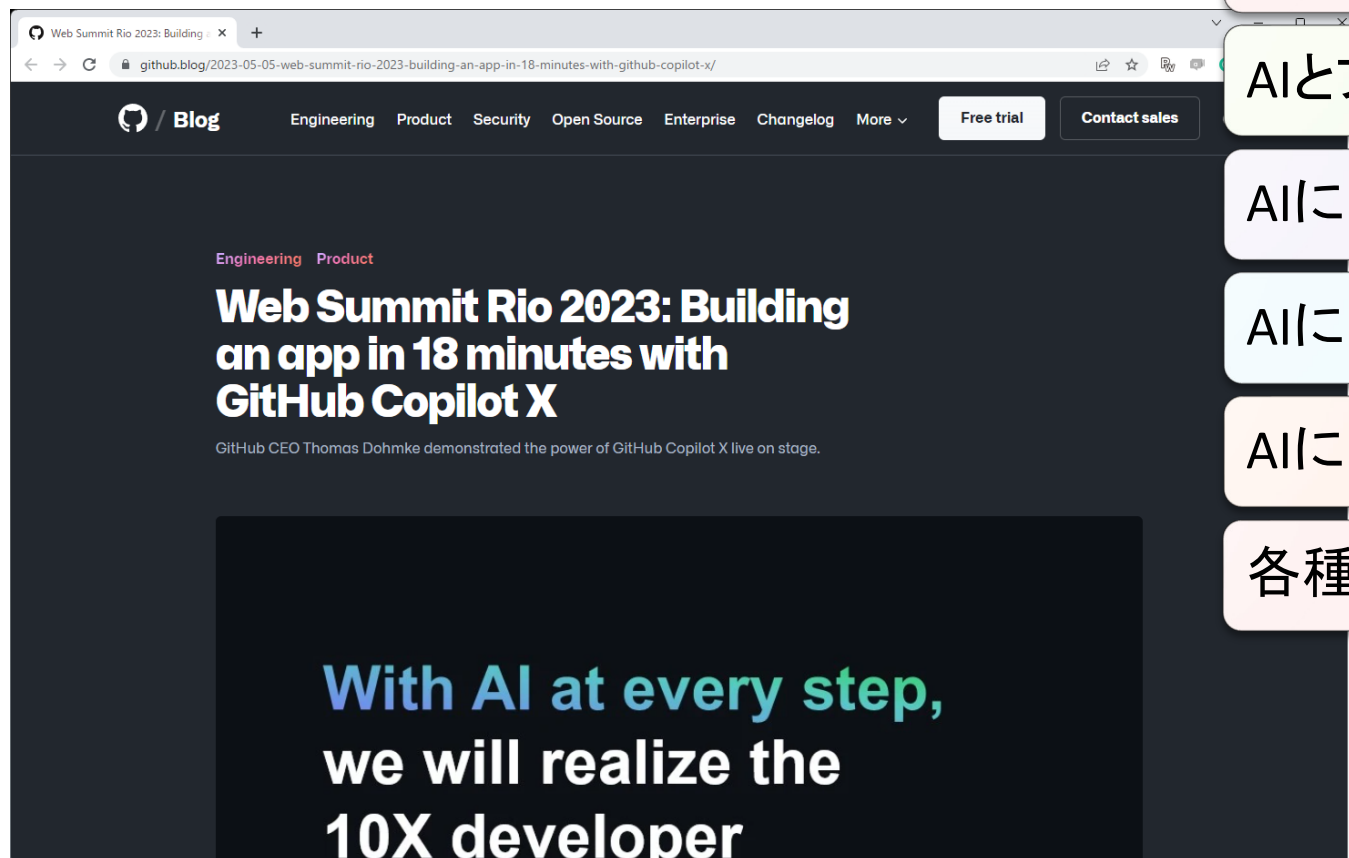
ソフトウェアツールを活用したDevOpsの実現



ソフトウェア開発において、ビルドやテストを頻繁に繰り返し行なうことにより問題を早期に発見し、開発の効率化や省力化、納期の短縮などを図る手法

ソフトウェア開発に おける生成AIの活用

GitHub Copilot X



AIを活用するプログラマの生産性は10倍に

AIとプログラマによるペアプログラミング

AIによるコード生成

AIによるコードレビュー

AIによるテストコード生成

各種のAI支援でソフトウェア開発に革命が

<https://github.blog/2023-05-05-web-summit-rio-2023-building-an-app-in-18-minutes-with-github-copilot-x/>

GitHub Copilot + Visual Studio Codeによるプログラミング支援

コード補完

- ステップごとにコードの検証が必要
- 補完パネルの表示により, 提案を一覧表示

人間の意図を柔軟に反映させたプログラムが生成できる

チャット

- プログラム全体を生成し, 生成したコードの解説を表示
- エラーについての解説や修正方法の提案

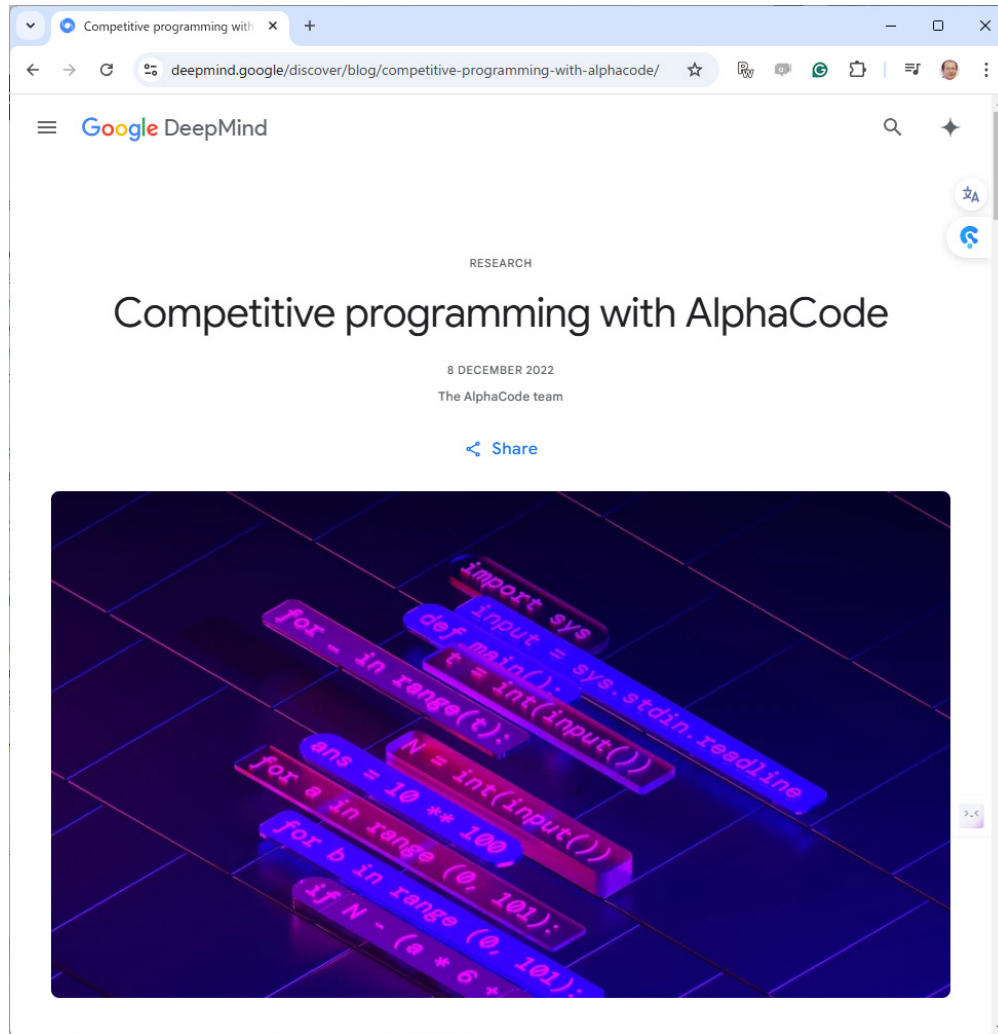
コード理解, レビュー, デバッグ支援などに有用

インラインチャット

- コードの一部を選択し, /explainコマンドを入力すると入力フィールドの下部に解説を表示

エディタから離れずにチャットを利用できるため, 作業効率が向上

AlphaCode (Google DeepMind)



競技プログラミングの問題を解決

自然言語で記述された問題文からプログラムを生成

AlphaCode 2 (2024)はGemini Proに基づくAIシステム

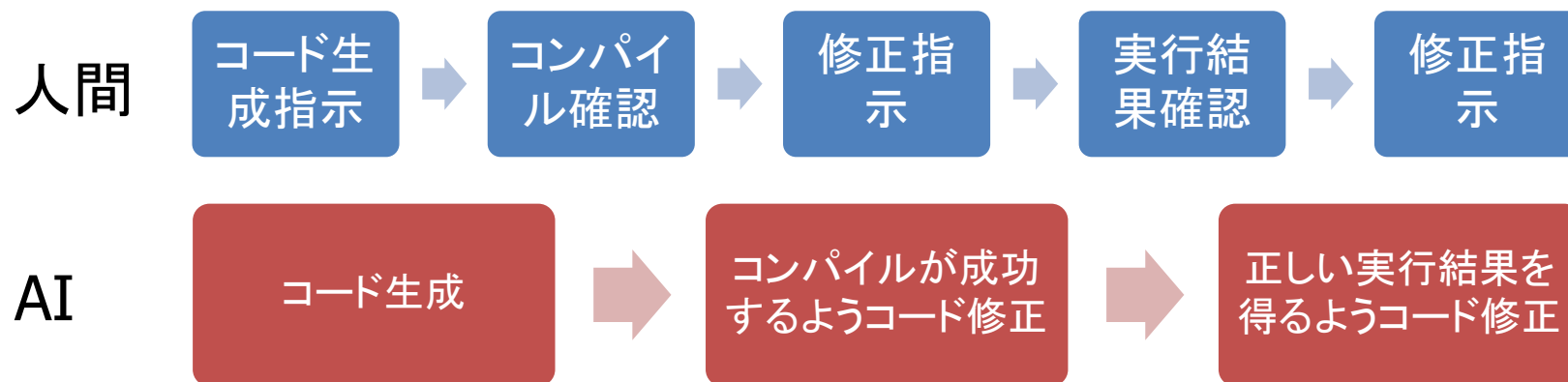
人間の競技者の85%を上回る性能を達成

プログラミング手順の変化

従来のプログラミング手順



今後想定されるプログラミング手順



人間の役割
が根本的に
変化

ソフトウェアの企画プロセス

問題を見つける

問題を定義する

問題の原因を探る

問題を解決するアイデアを探す

アイデアを改善する

生成AIを用いた思考の壁打ち

The screenshot shows a web browser with multiple tabs, all titled "生成AIを活用したプログラミング教育". The active tab displays a ChatGPT interface. The user's question is: "最近のソフトウェア開発では、アジャイル開発プロセスが標準になりつつあり、DevOpsのように各種のソフトウェアツールを用いることも普通に行われるようになっていきます。上記のようなプログラミングきょういくは、こうした変化にも対応しやすくなるのが期待できそうです。この考えは妥当でしょうか？". The AI's response is: "はい、この考えは妥当です。生成AIを活用したプログラミング教育は、アジャイル開発プロセスやDevOpsなどの現代的なソフトウェア開発手法に対応しやすくする要素を多く含んでいます。以下にその理由を具体的に示します。"

1. アジャイル開発プロセスへの対応

インクリメンタルかつ反復的な開発

- **生成AIの活用**：生成AIツールを使ってコードの自動生成やリファクタリングを行うことで、短期間で動作するプロトタイプを作成し、反復的に改良するアジャイルのサイクルに適応できます。
- **フィードバックの迅速化**：AIツールはリアルタイムでフィードバックを提供するため、学生が迅速にコードを修正・改善し、インクリメンタルな開発をサポートします。

コラボレーションとコミュニケーション

- **ペアプログラミングの支援**：生成AIツールはペアプログラミングの際にも役立ち、学生同士の協力を促進し、より質の高いコードを共同で作成することができます。

ChatGPT にメッセージを送信する

Get citation

ChatGPT の回答は必ずしも正しいとは限りません。重要な情報は確認するようにしてください。

研究テーマの企画例

KAKEN

研究課題をさがす

研究者をさがす

前のページに戻る

生成AIによるシステム開発の実践

研究課題/領域番号

24K06418

研究種目

基盤研究(C)

配分区分

基金

応募区分

一般

審査区分

小区分09080:科学教育関連

研究機関

佐賀大学

研究代表者

大月 美佳 佐賀大学

研究分担者

掛下 哲郎 佐賀大学

研究期間 (年度)

2024-04-01 - 2025-03-31

研究課題ステータス

交付 (2024年度)

配分額 *注記

4,550千円 (直接経費: 3,500千円、間接経費: 1,050千円)
2026年度: 1,300千円 (直接経費: 1,000千円、間接経費: 300千円)
2025年度: 1,430千円 (直接経費: 1,100千円、間接経費: 330千円)
2024年度: 1,690千円 (直接経費: 1,400千円、間接経費: 420千円)

キーワード

ソフトウェア技術者教育 / 生成AI / ヘアプログラミング

研究開始時の研究の概要

近年普及した生成AIによる開発現場の変革。本研究課題では、研究

KAKEN

研究課題をさがす

研究者をさがす

KAKENの使い方

日本語

前のページに戻る

生成AIによるプログラミング教育のパラダイム転換と教育支援ツールの研究開発

研究課題

サマリー

研究課題/領域番号

24K06431

研究種目

基盤研究(C)

配分区分

基金

応募区分

一般

審査区分

小区分09080:科学教育関連

研究機関

熊本高等専門学校

研究代表者

村田 美友紀 熊本高等専門学校, 拠点化プロジェクト系情報セキュリティグループ, 教授 (50290838)

研究分担者

掛下 哲郎 佐賀大学, 理工学部, 准教授 (10214272)
大月 美佳 佐賀大学, 理工学部, 講師 (20315138)
嘉藤 直子 有明工業高等専門学校, 一般教育科, 教授 (70280485)

研究期間 (年度)

2024-04-01 - 2027-03-31

研究課題ステータス

交付 (2024年度)

配分額 *注記

4,550千円 (直接経費: 3,500千円、間接経費: 1,050千円)
2026年度: 1,300千円 (直接経費: 1,000千円、間接経費: 300千円)
2025年度: 1,430千円 (直接経費: 1,100千円、間接経費: 330千円)
2024年度: 1,620千円 (直接経費: 1,400千円、間接経費: 420千円)

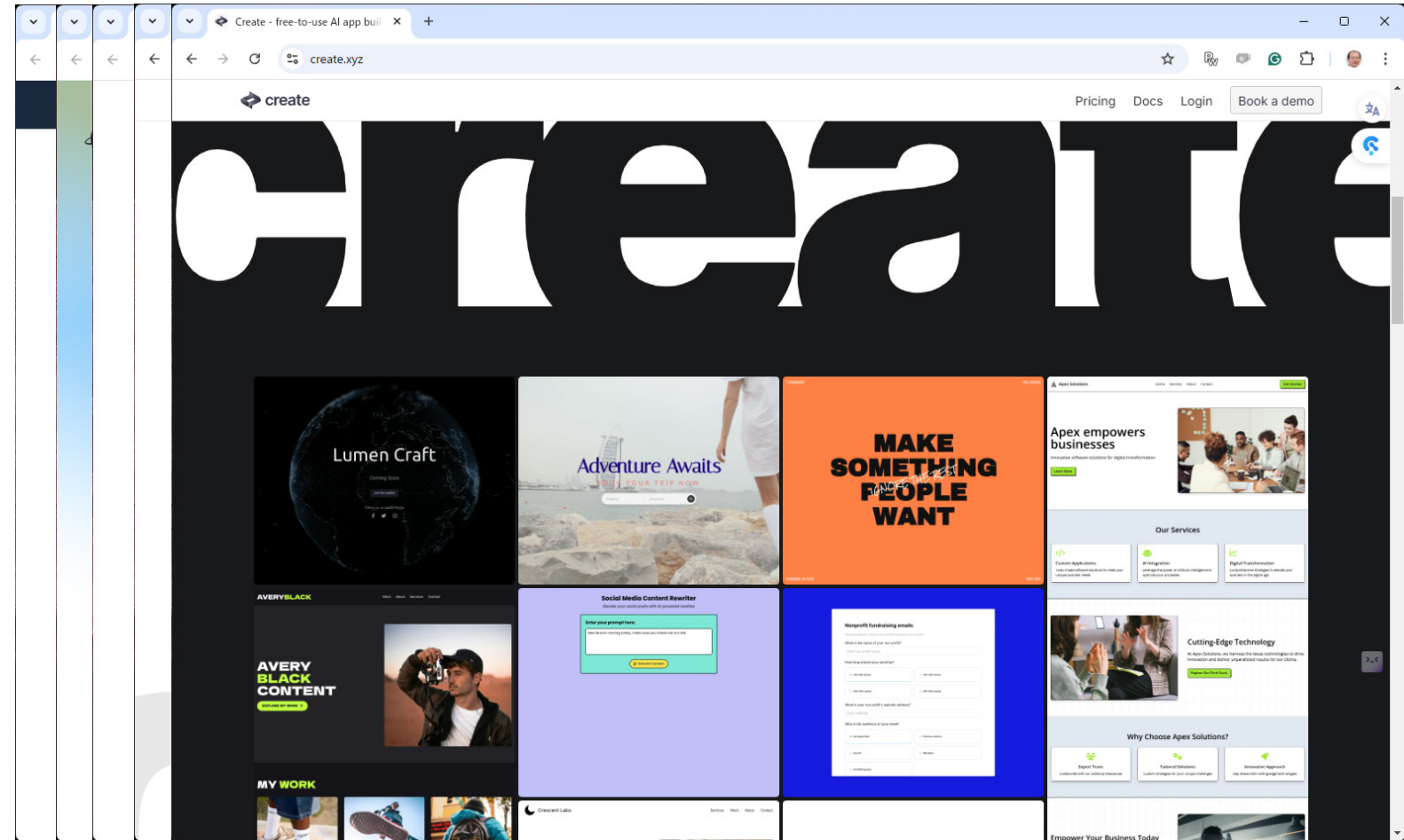
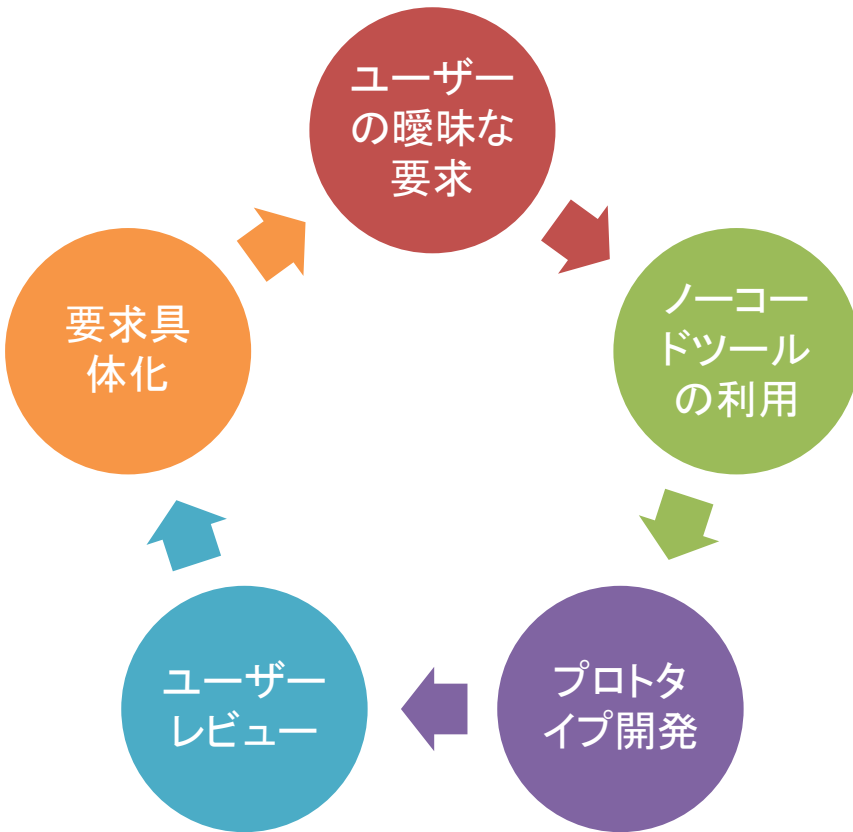
キーワード

プログラミング教育 / 生成AI / ヘアプログラミング

研究開始時の研究の概要

生成AIは急速に普及しており、特にソフトウェア開発においては、設計からテストに至る工程の中でChatGPTのようなテキスト生成AIを活用した自動化が進行中である。これに伴い、大学や高専におけるプログラミング教育も人間が直接プログラミングすることを前提とした従来の教育からのパラダイム転換が求められている。

ノーコードツールを活用した仕様策定



パターン言語の活用



各種パターンをAI
に学習させる



問題を提示して提
供可能なパターン
をAIに提示させる



具体的なソフト
ウェア設計案をAI
に提示させる

GitHub Copilot Workspace

https://www.publickey1.jp/blog/24/githubcopilot_workspaceai.html

Publickey Enterprise IT Technologies

GitHub, 「Copilot Workspace」テクニカルレビューを開始。ほとんど全ての開発工程をAIで自動化

2024年4月30日

GitHubは、昨年（2023年）11月にCopilotの将来像として紹介したAIによるプログラミング支援機能「Copilot Workspace」のテクニカルレビューを開始したと発表しました。

Copilot Workspace
A Copilot-native dev environment, designed for everyday tasks.

テクニカルレビューは上記のCopilot Workspaceのページからウェイトリストボタンをクリックして申し込みます。

Copilot Workspaceはほとんど全ての工程を自動化

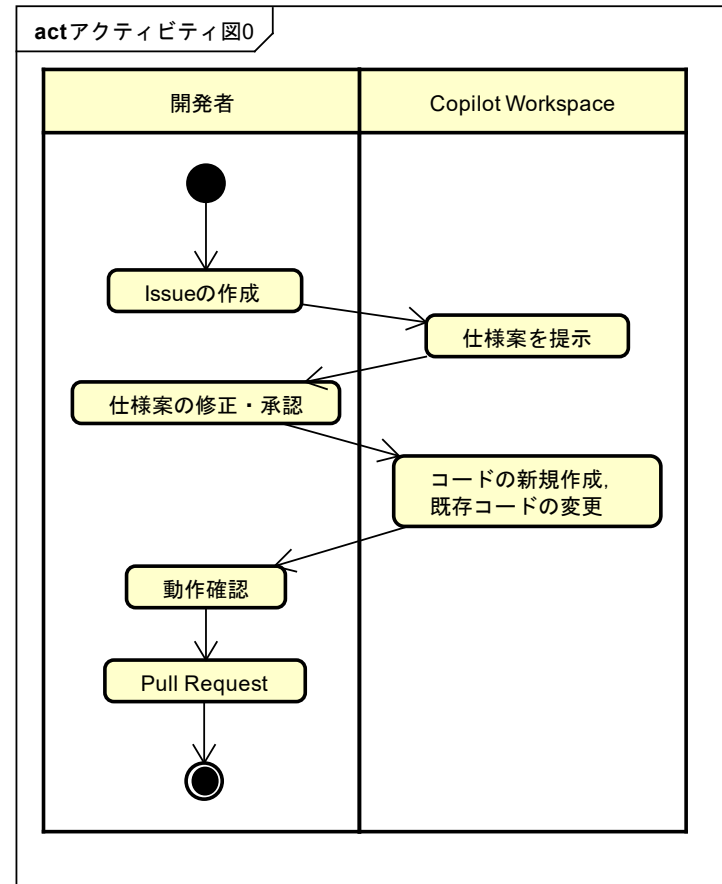
Copilot Workspaceは、自然言語で書かれたIssue（課題）を基に、Copilotが仕様案と実装計画を示し、コーディングや既存のコードの修正を行い、ピ

タグクラウド

- クラウド
 - AWS / Azure / Google Cloud
 - クラウドネイティブ / サーバレス
 - クラウドのシェア / クラウドの障害
- コンテナ型仮想化
- プログラミング言語
 - JavaScript / Java / .NET
 - WebAssembly / Web標準
 - 開発ツール / テスト・品質
- アジャイル開発 / スクラム / DevOps
- データベース / 機械学習・AI
 - RDB / NoSQL
- ネットワーク / セキュリティ
 - HTTP / QUIC
- OS / Windows / Linux / 仮想化
- サーバ / ストレージ / ハードウェア
- ITエンジニアの給与・年収 / 働き方
- 殿堂入り / おもしろ / 編集後記
- 全てのタグを見る

帳票開発の工数、削減しませんか？
エンドユーザーがExcelで帳票をデザイン

VB-Report 11
ノーコード・ローコード帳票ツール付属
.NET5/.NET Framework対応 | サーバー/クラウド対応
Advance Software



生成AIを利用したソフトウェアテスト

The screenshot displays three browser tabs illustrating AI-powered software testing solutions:

- GitHub Copilot**: A Japanese article titled "GitHub Copilotを使ったテストコード" (Test code using GitHub Copilot). It describes how Copilot can automatically generate test code based on comments, reducing the time and effort required for test development. A table of steps is provided:

手順
1. テスト対象のコードを記述する
2. テストコードのコメントを記述する
3. Copilotにテストコードの生成を依頼する
4. 提案されたテストコードを確認・修正
5. テストコードを実行して検証する

- AppliTools**: The "AI-POWERED END-TO-END TESTING" platform. It emphasizes "Maximum Coverage" and "Minimum Time". Key features include:
 - Scale Test Creation
 - Automate Maintenance
 - Eliminate False PositivesButtons for "SCHEDULE A DEMO" and "REQUEST A FREE TRIAL" are visible.
- Testim**: The Tricentis Testim homepage. The headline is "Move faster and reduce costs with the power of AI". It states: "Accelerate test authoring, reduce maintenance, and release higher-quality apps faster." Buttons for "TRY TESTIM FOR FREE" and "WATCH THE VIDEO" are present. A large graphic shows a Salesforce logo on a circuit board. At the bottom, three benefits are listed:
 - Grow test coverage faster**: Build high-quality tests more
 - Eliminate costly maintenance**: AI and ML smart locators let
 - Identify and resolve problems**: Diagnose failed tests to findA chat bubble from "Ian from Tricentis" is visible in the bottom right corner.

ITベンダーの生成AI活用事例

会社名	生成AIの適用方針	生産性向上の目標値	利用する主な生成AI	導入部署・導入数
NTTデータ	要件定義からデプロイまでに活用。信金などの大規模システムを学習させマイグレーションにも利用する	2025年度に50%、2030年度に70%削減	GitHub Copilot など	約5000ライセンスを計画
NEC	実装から単体テストまでに活用。「生成AI活用ガイドライン」を作成し上流工程へシフトする。モイゼーションへの適用も検証中	20%～50%、平均30%の生産性向上を見込む	GitHub Copilot	2024年度から実践フェーズに入る。2024年2月時点の利用は100～200人
日立製作所	大規模システムに適用する。まずはGitHubでコーディングと単体テストの効率を高め、次に基本設計から総合テストにまでに広げる、ソリューション開発やマイグレーション、運用保守、品質保証も視野に入れる	2027年度に基本設計から総合テストまでを対象に30%の削減を目標にする	Azure OpenAI Service	研究所や事業部門などで活用。利用者数は不明
富士通	自社の標準開発基盤「Fujitsu Developers Platform (DP)」に生成AIを取り込み、仕様設計からプログラミング、テストまでに適用する	目標値はこれから決めるが、平均20%の生産性向上を見込む	Azure OpenAI Service	ソフトウェアオープンイノベーション事業本部では半数の約800人。全社の導入数は不明

生成AIで支援できそうなタスク

アイデア創
出

各種ドキュメ
ント作成

UIデザイン作
成

仕様書・設計
書レビュー

ソースコード
生成

ソースコード
レビュー

リファクタリン
グ

プロジェクト
管理

テストケース
生成

テストコード
生成

コードや文書
の内容把握

各種調査・質
問回答